

AI-Assisted Quantum Circuit Transpilation

Improving transpilation for square-lattice QPUs

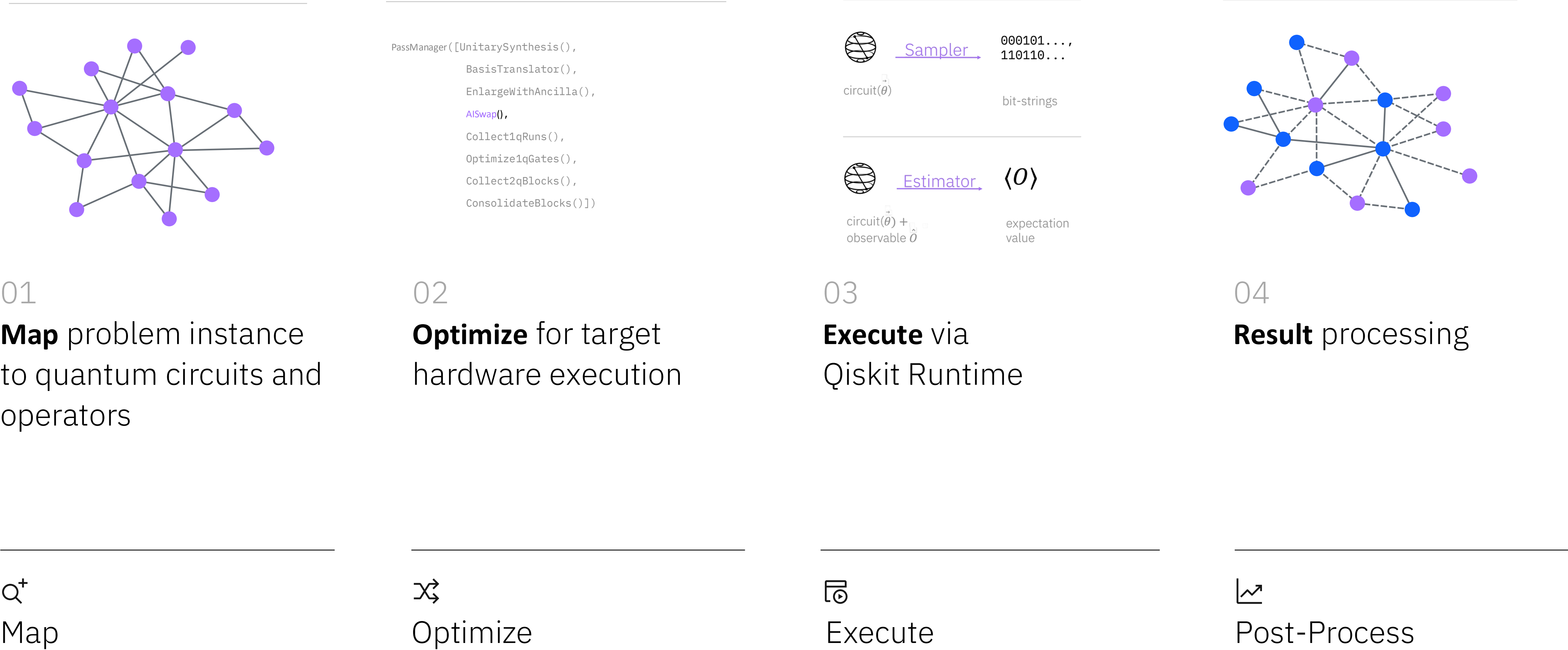
Duru Tomruk

IBM Quantum x DHBW Stuttgart

Built on the IBM Quantum “Circuit Transpilation” deck · template & original slides by Junye Huang (IBM)

Qiskit Pattern:

The anatomy of a quantum algorithm



Instruction set architecture

Quantum circuits come in many different shapes and forms; however, **not all of them can run directly on quantum hardware.**

To run, circuits must conform to the hardware's specific **Instruction Set Architecture (ISA)**. This kind of circuits are known as **ISA circuits**.

ISA constraints come in the form of:

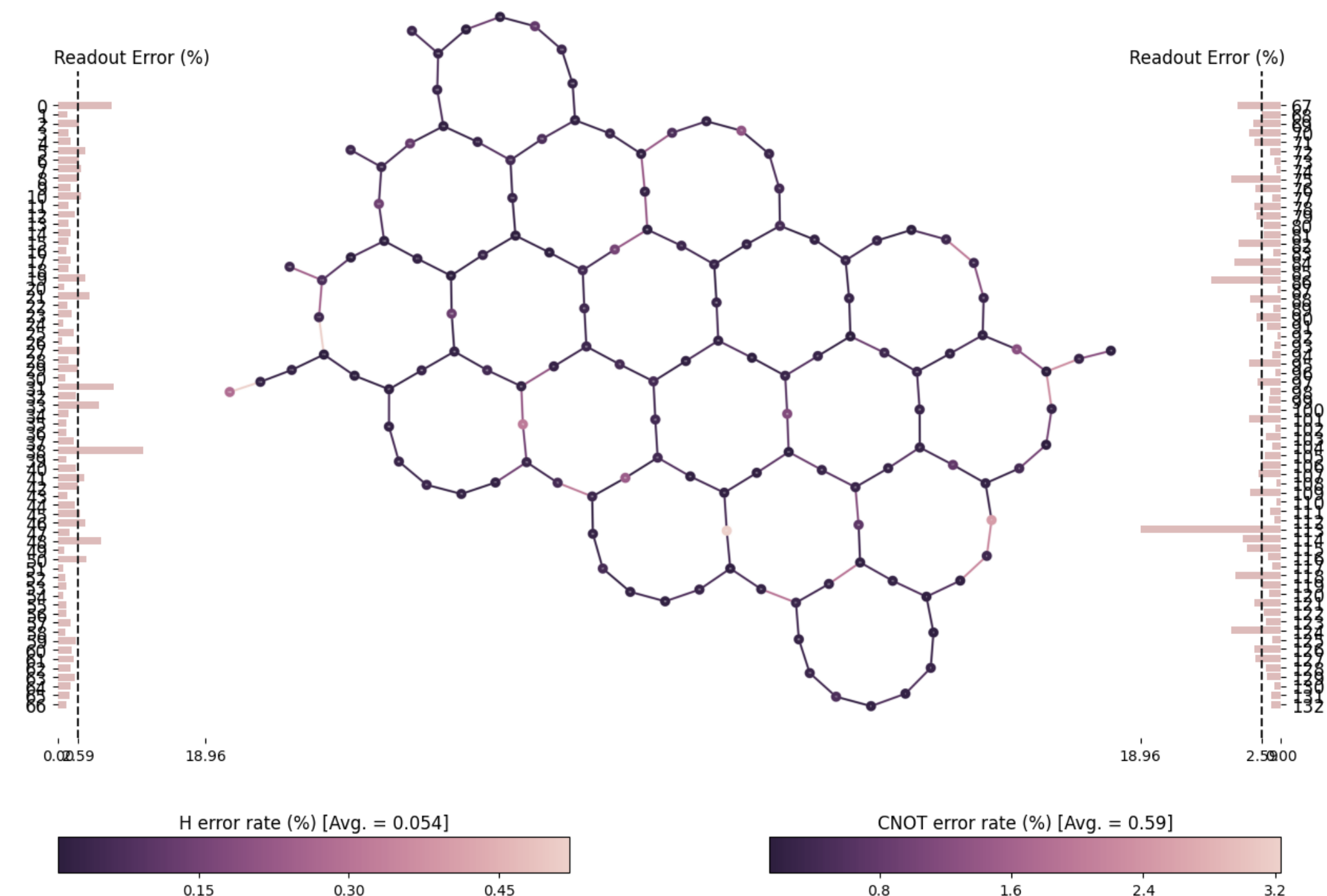
1. Gate types
2. Qubit connectivity



```
from qiskit_ibm_runtime import QiskitRuntimeService
from qiskit.visualization import plot_error_map
```

```
service = QiskitRuntimeService()
backend = service.least_busy()
```

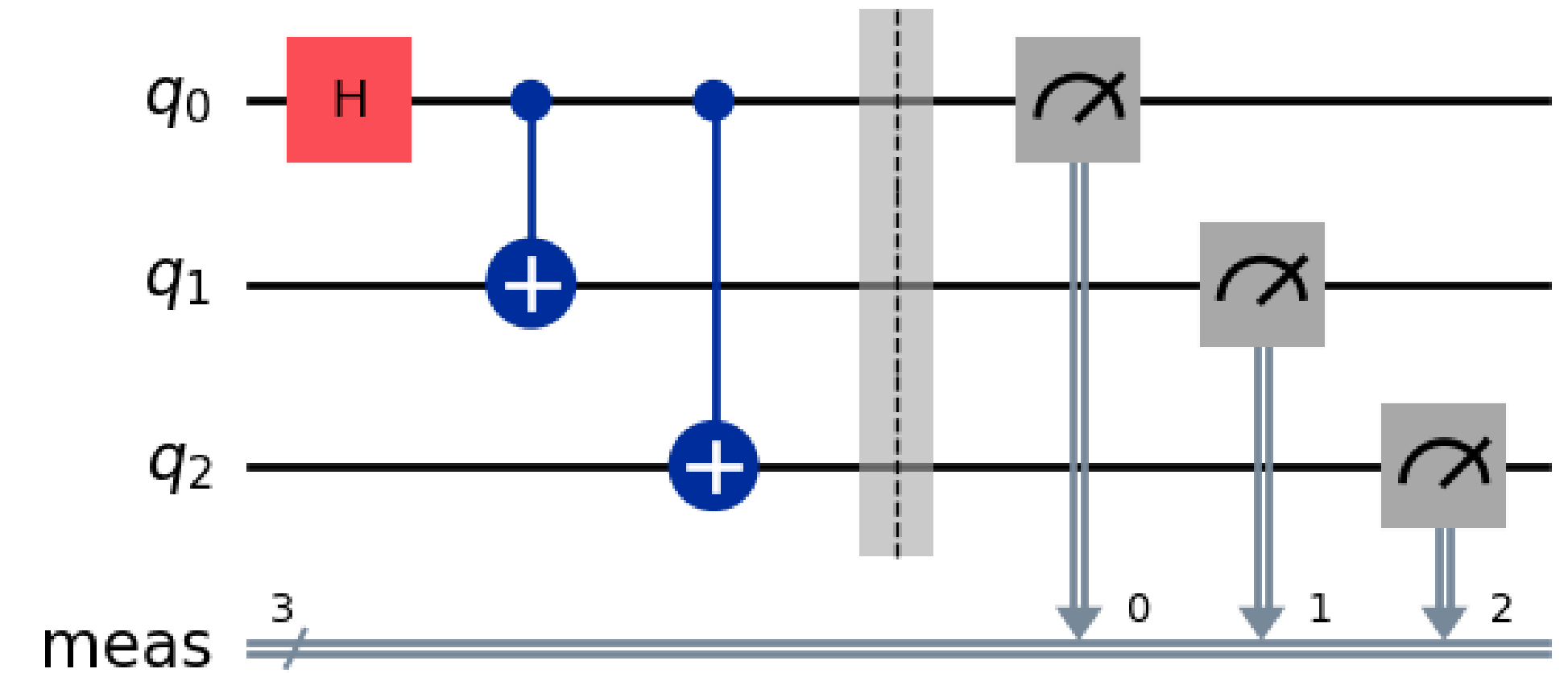
```
plot_error_map(backend, show_title=False)
```



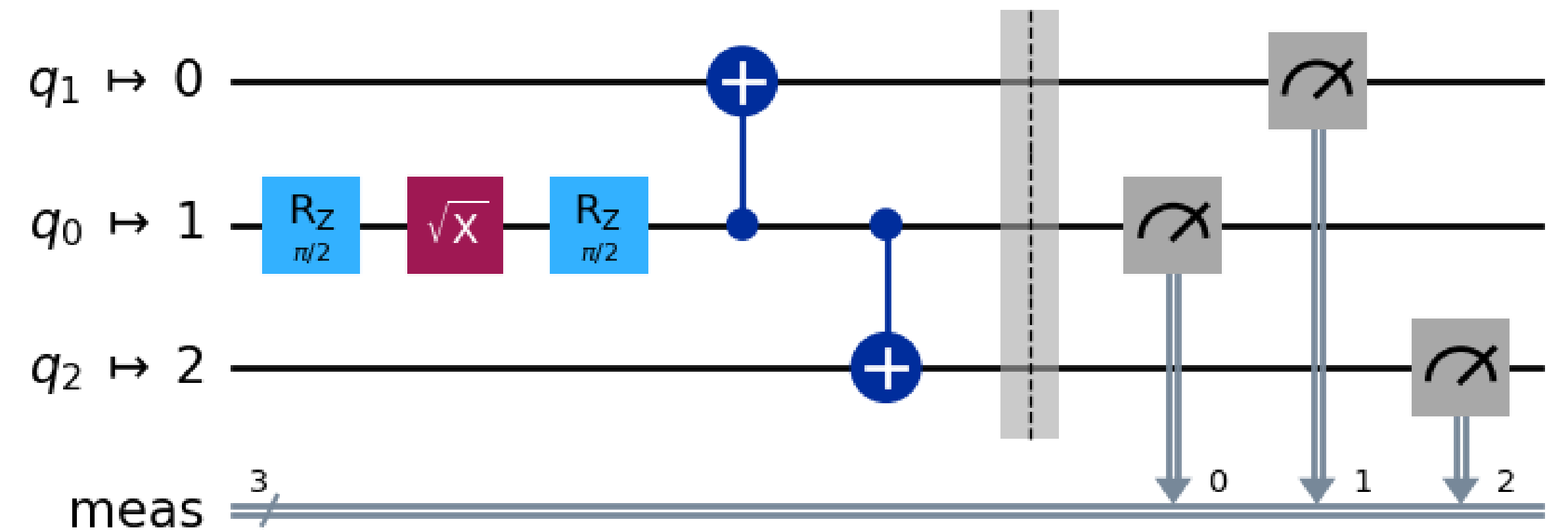
What is Transpilation?

An ISA is said to be [universal](#) if it can express any operation possible on a quantum computer.

The process of expressing a quantum circuit in ISA form is known as [transpilation](#), since the (circuit) level of abstraction is preserved.



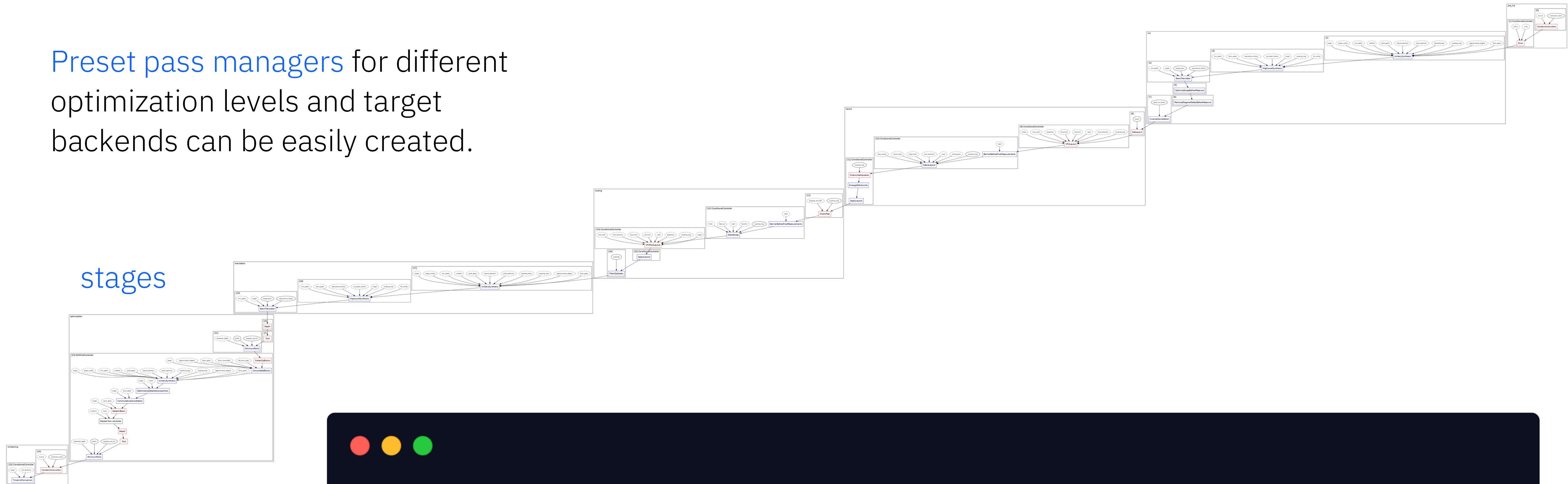
Global Phase: $\pi/4$



Transpiler pipeline

Preset pass managers for different optimization levels and target backends can be easily created.

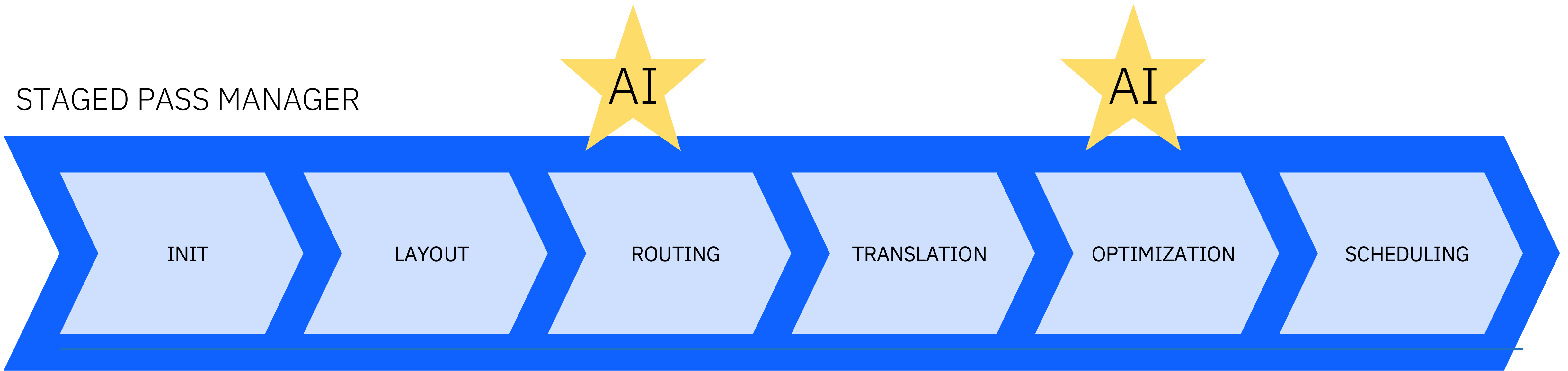
stages



```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

pm = generate_preset_pass_manager(optimization_level=3, backend=backend)
pm.draw()
```

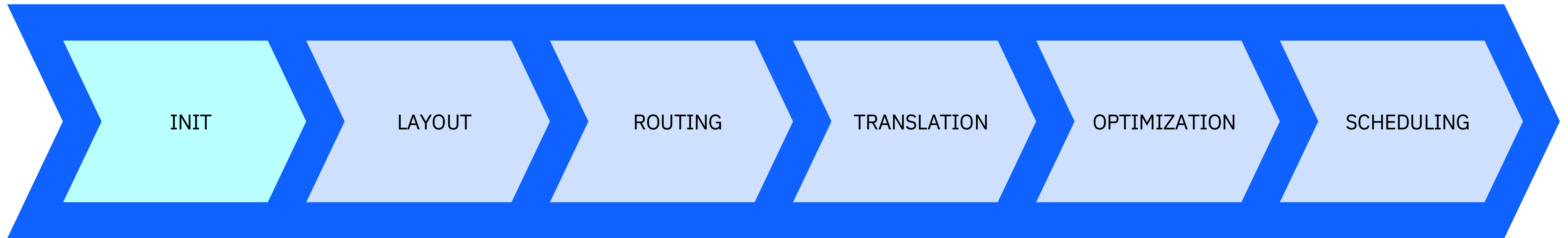
Transpiler stages



We will watch all 6 happen to one tiny circuit, the Bell state.

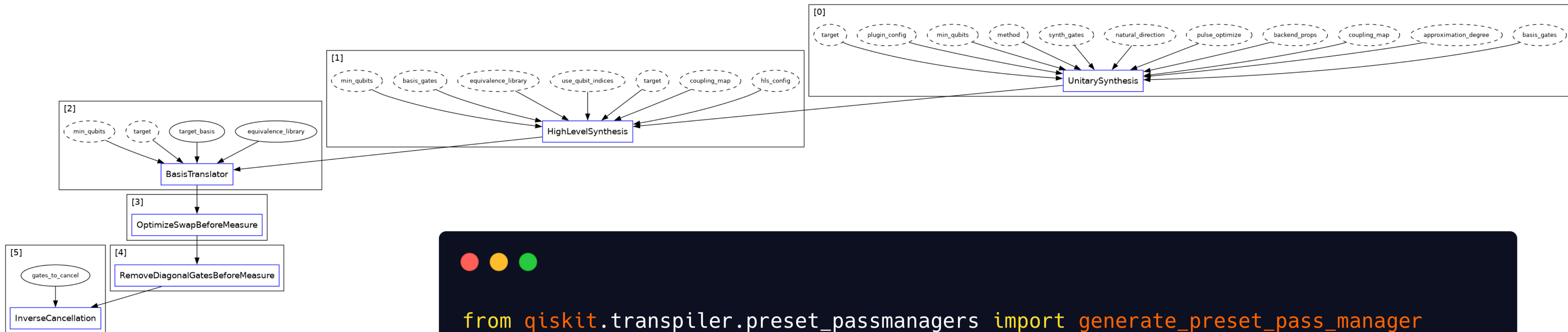
Transpiler stages

STAGED PASS MANAGER



Init stage

1. Unitary synthesis
2. High level synthesis
3. Basis translation
4. Optimize SWAP before measurement
5. Remove diagonal gates before measurements
6. Inverse cancelation

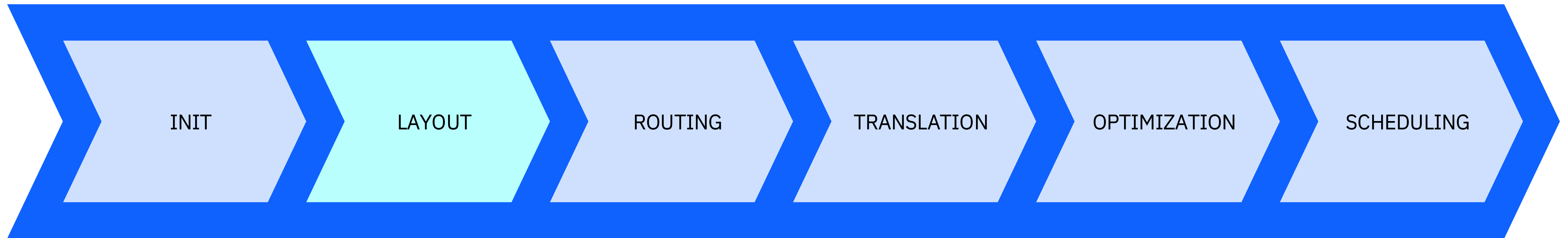


```
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

pm = generate_preset_pass_manager(optimization_level=3, backend=backend)
pm.init.draw()
```


Transpiler stages

STAGED PASS MANAGER

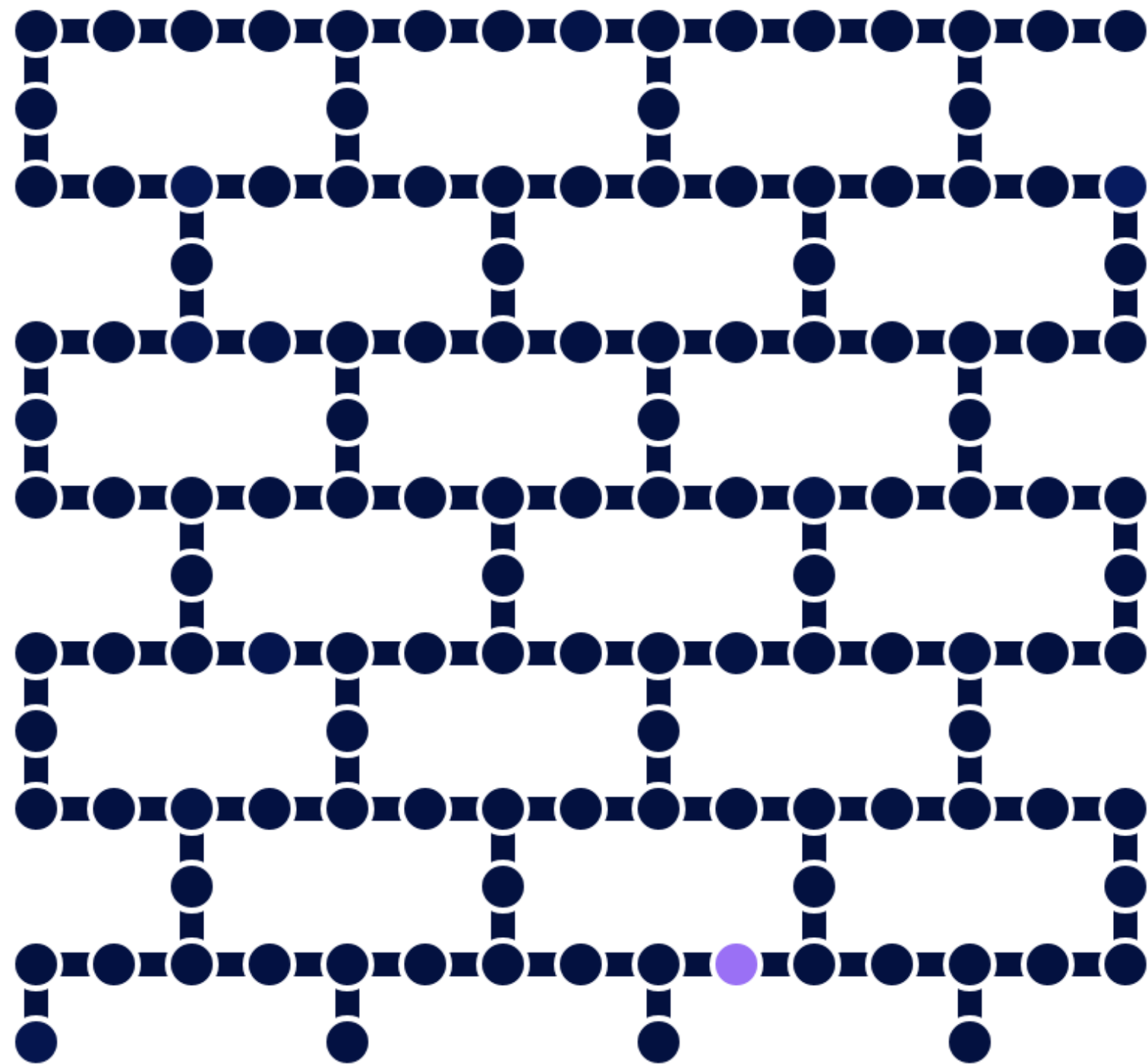


Layout stage: Logical qubits $\rightarrow$ physical qubits

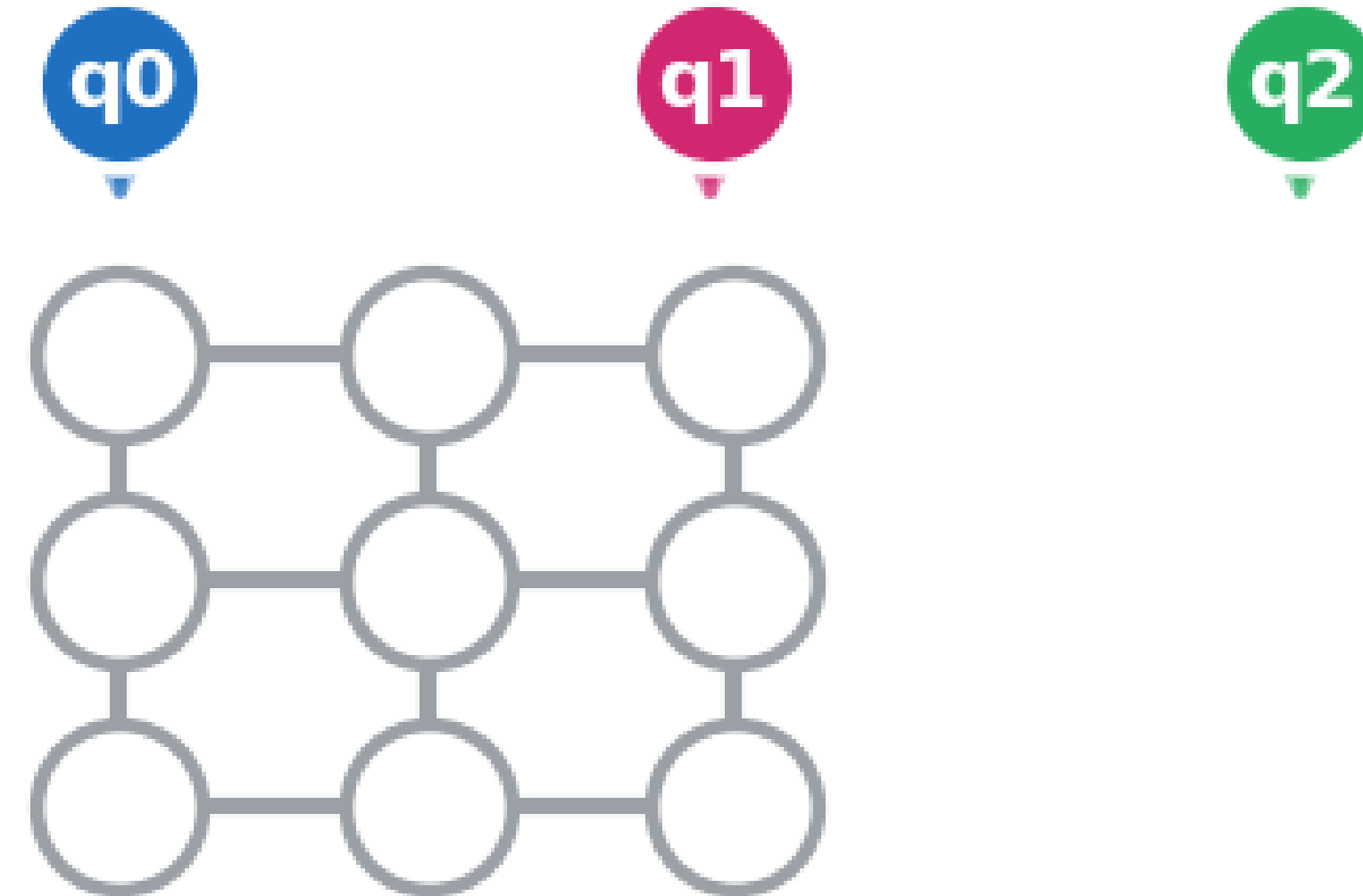
Logical qubits = your variables, Physical qubits = real spots on the chip

Not every pair is wired $\rightarrow$ insert SWAPs to move data

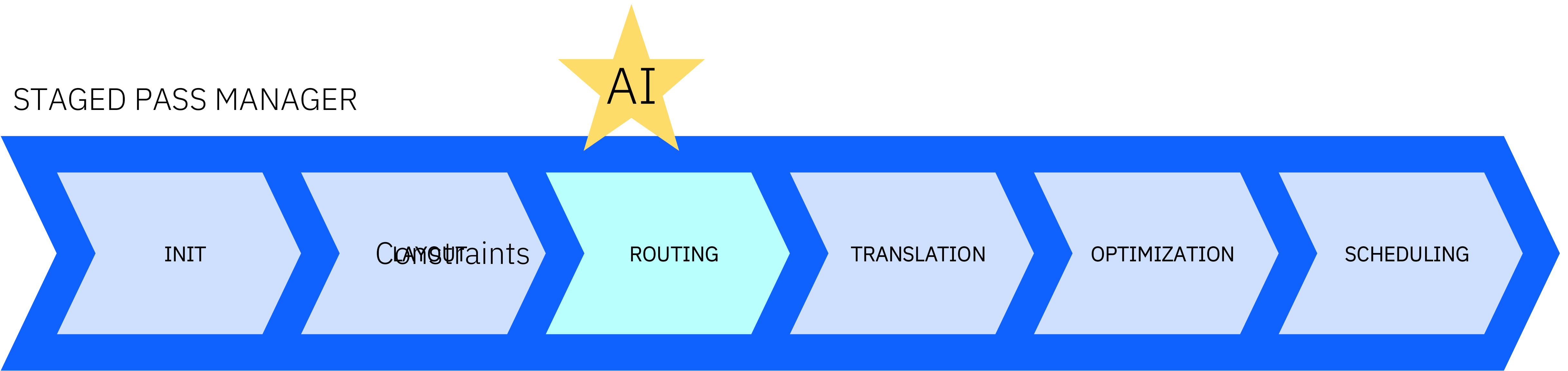
Each SWAP = 3 CNOTs $\rightarrow$ keep them few



logical qubits $\rightarrow$ physical qubits



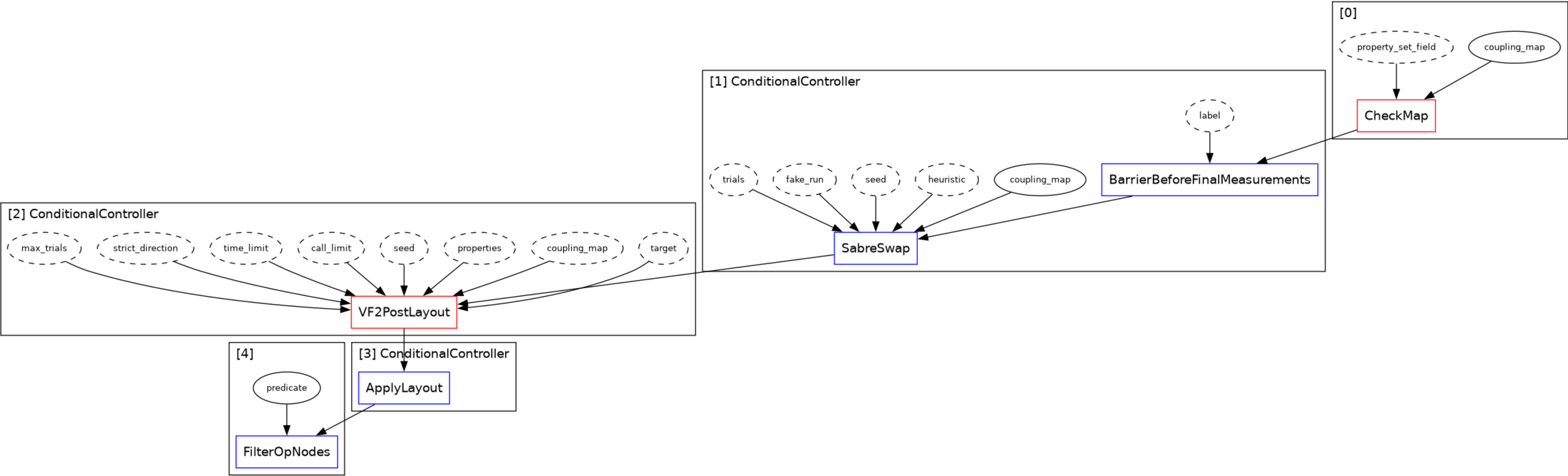
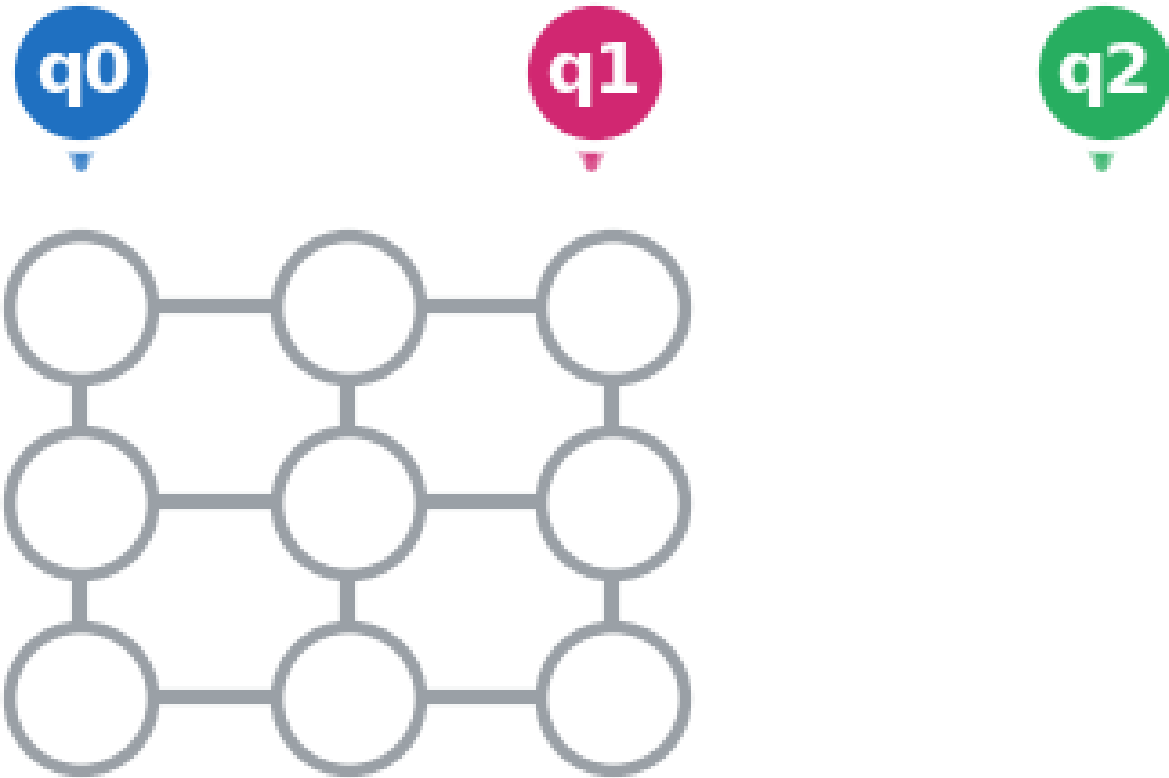
Transpiler stages



Routing stage

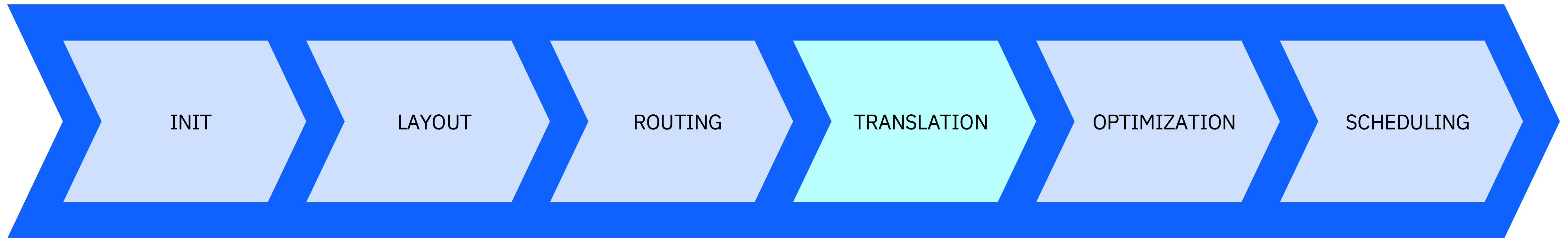
- 1. Check if routing is needed
- 2. SABRE Swap (AI replaces this with AI Routing)
- 3. Choose the best isomorphic subgraph in terms of error rates.

logical qubits → physical qubits



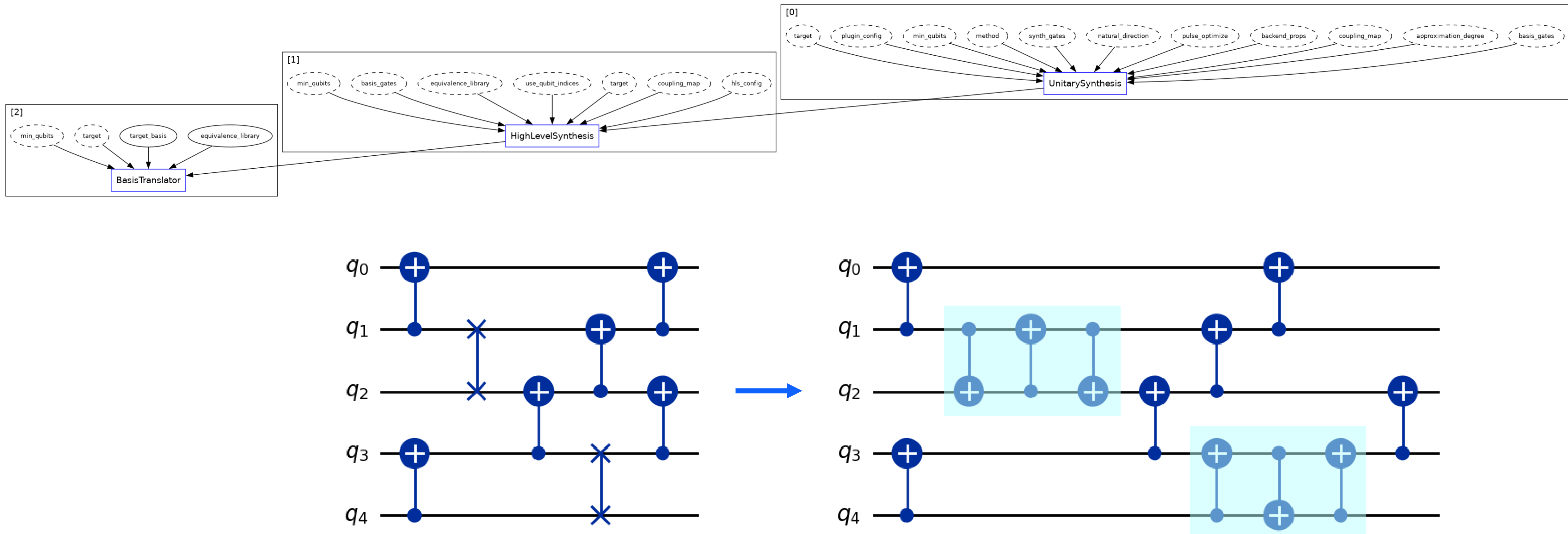
Transpiler stages

STAGED PASS MANAGER



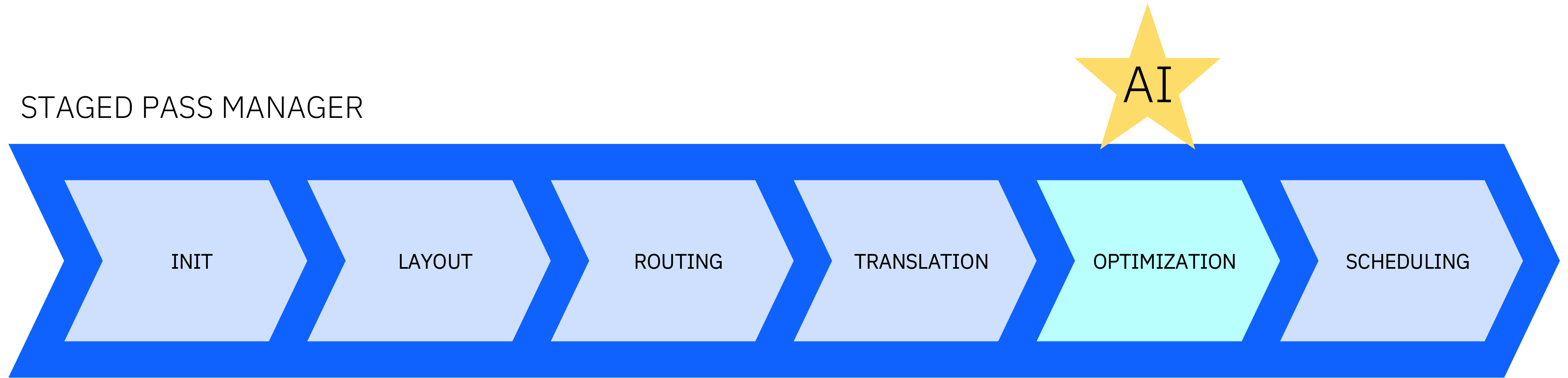
Translation stage

At this point we can translate all remaining operations into the hardware's ISA.



Transpiler stages

STAGED PASS MANAGER



Optimization stage: Transpilation as a game

Optimization is performed for one- and two-qubit blocks until a minimum is reached.

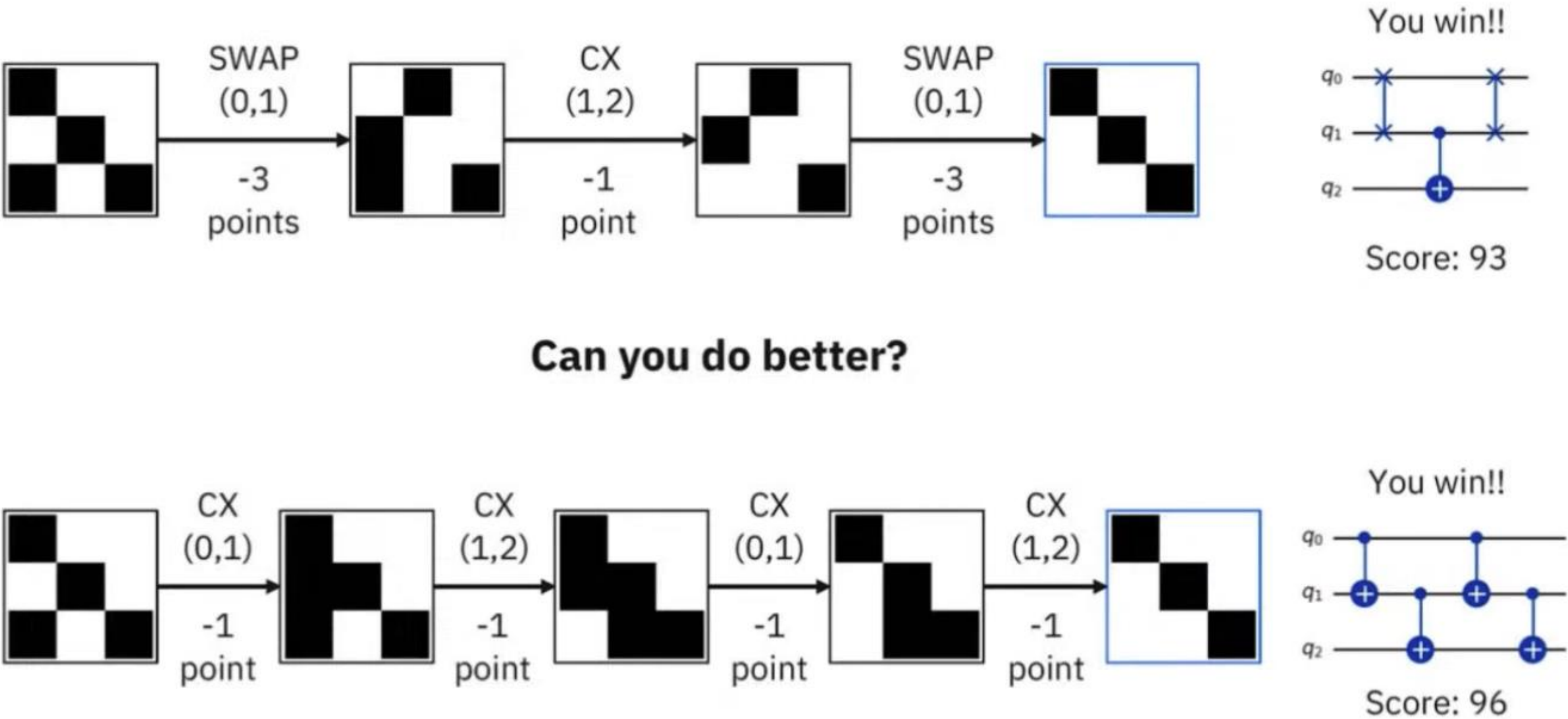
Board: the operation still left to build (a binary matrix)

Moves: add one allowed CNOT (only on wired qubit pairs)

Goal: reduce the board to “done” (identity)

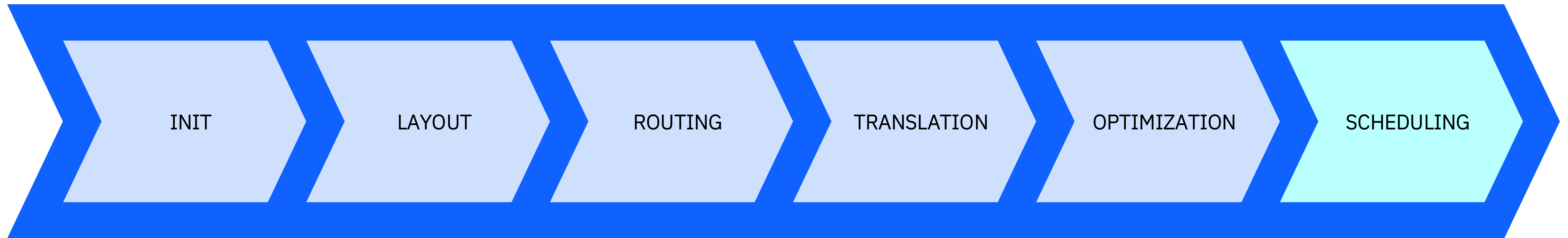
Score: cheaper circuit = higher score (2-qubit gates cost more)

(1 SWAP = 3 CNOTs)



Transpiler stages

STAGED PASS MANAGER



(Optional)

DEMO 1: The Bell state through 6 stages

Bell state = just 3 operations → easy to see every stage

Watch: depth 3 → grows (routing/translation) → shrinks (optimization)

Backend: ibm_marrakesh

► Switch to code

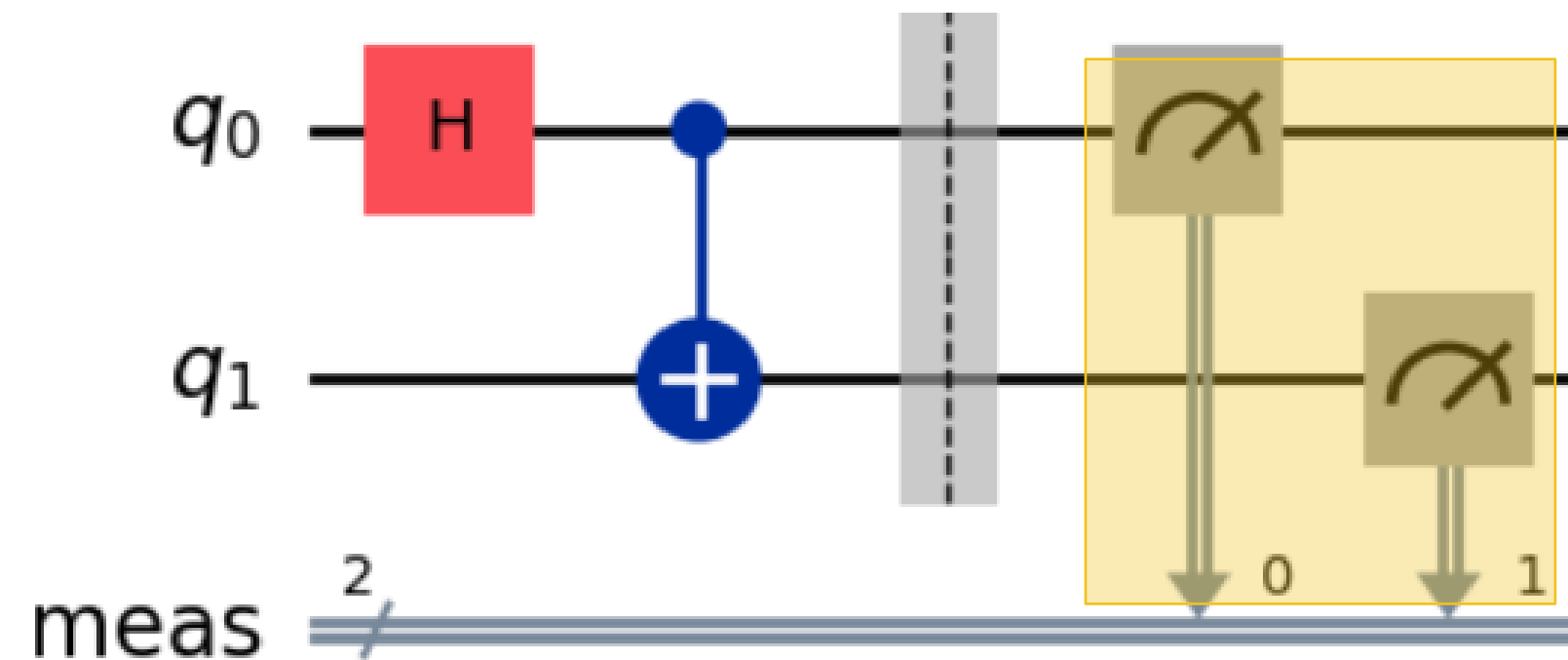


Fig. 3: Bell circuit after Stage 1. Gate count: H:1, CNOT:1, measure:2; depth 3.

H:1, CNOT:1, measure:2, depth 3

AI transpiler

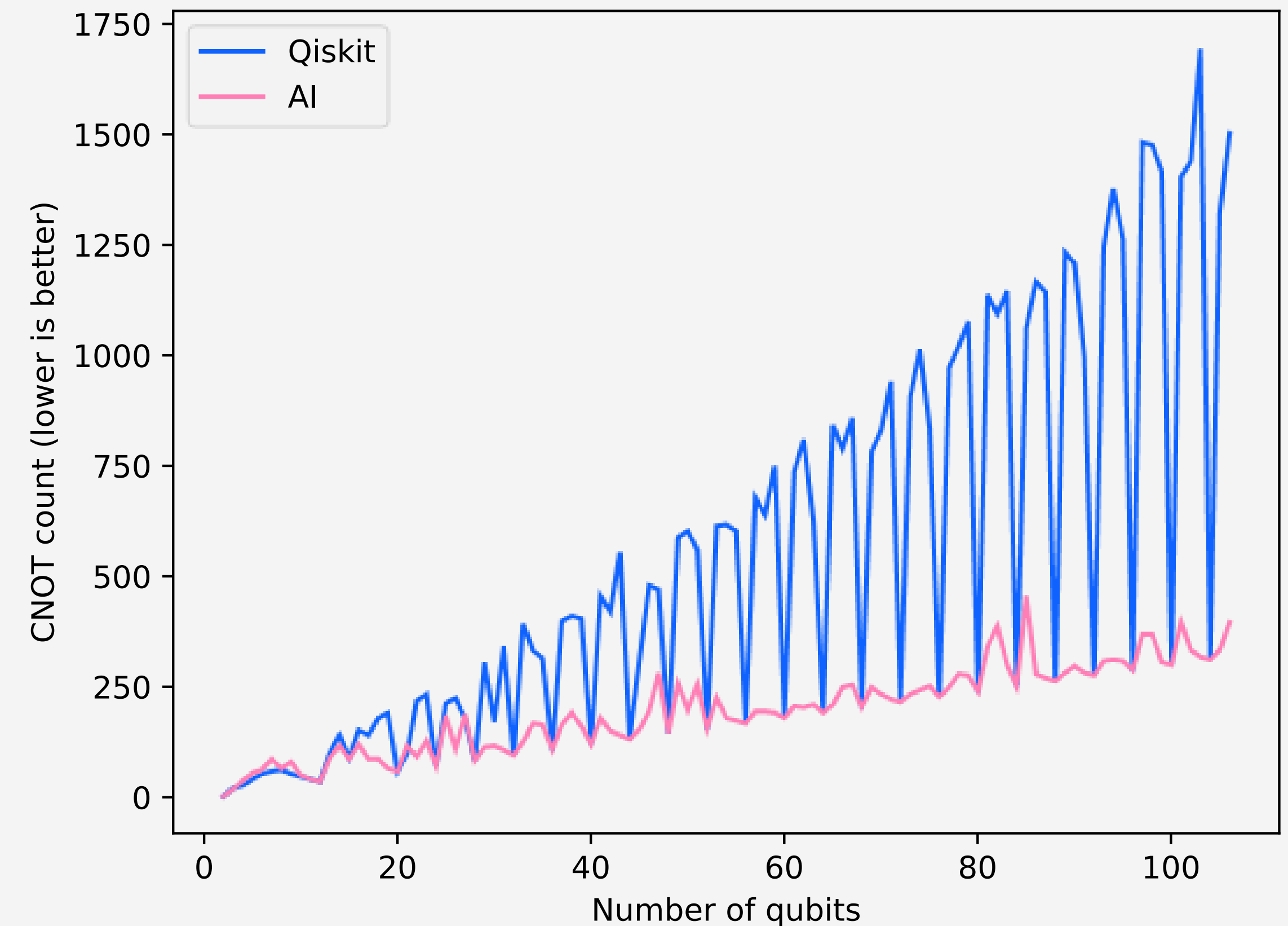
Reinforcement Learning (PPO) learns the policy

It plays the game millions of times

20-50% improvement in depth / CNOT count
vs Qiskit

Catch: each model is trained for ONE coupling
map

EfficientSU2 Circuits after Transpilation



DEMO 2: The AI transpiler, by the numbers

Classical Transpiler vs AI Transpiler, transpilation “game” in code

► Switch to code

arXiv:2405.13196v2 [quant-ph] 26 Feb 2025

Practical and efficient quantum circuit synthesis and transpiling with Reinforcement Learning

David Kremer,^{1,*} Victor Villar,¹ Hanhee Paik,¹ Ivan Duran,¹ Ismael Faro,¹ and Juan Cruz-Benito^{1,†}

¹*IBM Quantum, IBM T.J. Watson Research Center, Yorktown Heights, NY 10598*

This paper demonstrates the integration of Reinforcement Learning (RL) into quantum transpiling workflows, significantly enhancing the synthesis and routing of quantum circuits. By employing RL, we achieve near-optimal synthesis of Linear Function, Clifford, and Permutation circuits, up to 9, 11 and 65 qubits respectively, while being compatible with native device instruction sets and connectivity constraints, and orders of magnitude faster than optimization methods such as SAT solvers. We also achieve significant reductions in two-qubit gate depth and count for circuit routing up to 133 qubits with respect to other routing heuristics such as SABRE. We find the method to be efficient enough to be useful in practice in typical quantum transpiling pipelines. Our results set the stage for further AI-powered enhancements of quantum computing workflows.

I. INTRODUCTION

The field of quantum computing has grown into a community that includes domain experts in a wide range of various disciplines such as chemistry and Artificial Intelligence (AI) from industry, government and academia. With the rapidly developing technologies, quantum computing is now accessible by users via cloud where users are exploring how to utilize quantum computing as a tool for their goals. In 2023, IBM has demonstrated that with 127-qubit Eagle processor and error mitigation techniques, we can execute quantum circuits outside the reach of classical exact computing methods – the beginning of quantum utility [1]. Now quantum computing is evolving to expand its computational power by integrating with classical computing resources. Exploiting classical resources will allow us to extend the computational capacity that the current quantum hardware can offer and increase the workloads that are required for practical quantum computing. The future of computing would be a computing system where the orchestration between classical computing resources such as CPUs and GPUs together with multiple quantum processors (QPUs) [2, 3], forms a composite architecture that allows for scaling quantum computation to sizes amenable

to industry scale problems on the order of tens of thousands of qubits [4].

The integration of AI tools is considered a critical path to advance in several scientific fields [5–11]. In particular, we observe AI has the potential to help leverage the full potential of quantum computing [12]. As the sizes of quantum circuits input to quantum computing systems continue to grow in the era of quantum utility, there is a need for smarter and more effective methods to enhance the quantum computation workflow. The integration of classical computing resources, such as GPUs and CPUs, with QPUs will pave the way for new frontiers in computing, and the collaboration between AI and quantum computing will be critical in realizing this potential.

Quantum circuit transpilation and optimization are central components of quantum computing workflows. Similar to compilers in classical computing, transpilers map logical quantum circuits to the instructions present physical quantum devices, and allow quantum circuit developers to focus on the quantum algorithms rather than specific details of the hardware. High-quality transpilation of quantum circuits (minimizing the overhead introduced in this mapping) is an important goal for the field of quantum computing in general, but especially relevant for near-term quantum computing hardware, where even small improvements on the transpiled circuit sizes can lead to important reductions in the noise present in the results.

Circuit synthesis is a useful task within transpiling workflows and consists on generating a quantum circuit, within a given set of gates, that implements a high-level description of a quantum operator. A typical workflow with circuit synthesis is to re-synthesize parts of a circuit to see if a “better” circuit can be found and replace the original circuit parts if successful.

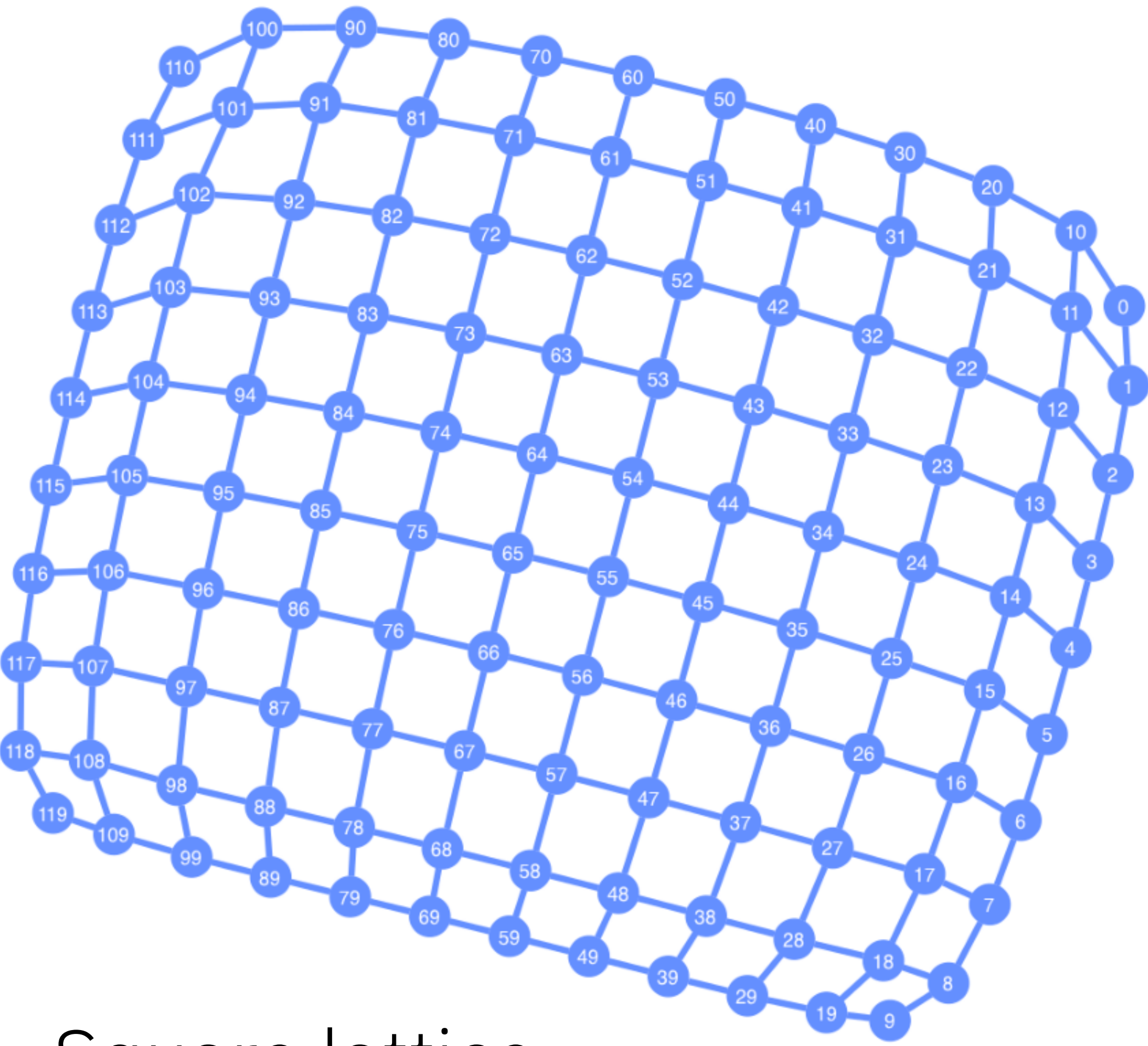
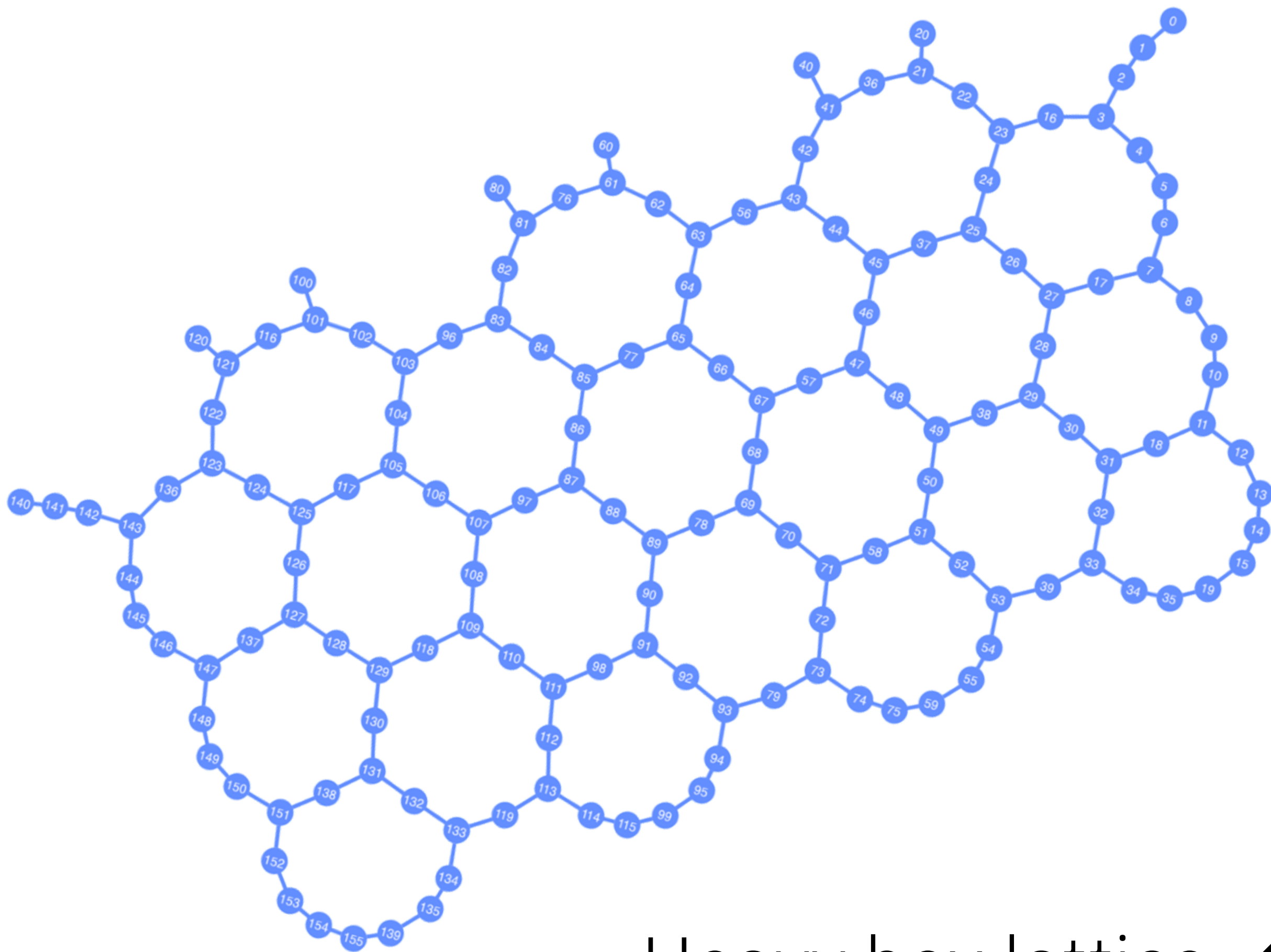
The practical usefulness of such workflow relies on having access to synthesis methods that can produce optimal or near-optimal circuits, with a reasonable amount of computational resources. Circuit synthesis methods typically fall in three categories:

- **Heuristic methods:** Heuristic methods are typically fast, but often produce sub-optimal circuits and work with a specific gate set such that changing a gate set requires modifying the algorithm or incurs a further translation cost.
- **Databases of optimal circuits:** Databases of optimal circuits can provide optimal circuits quickly, but take a long time to generate and require large data storage. Also fixed to specific gate sets.

* david.kremer@ibm.com

† juan.cruz.benito@ibm.com

Why this research: new chip architecture

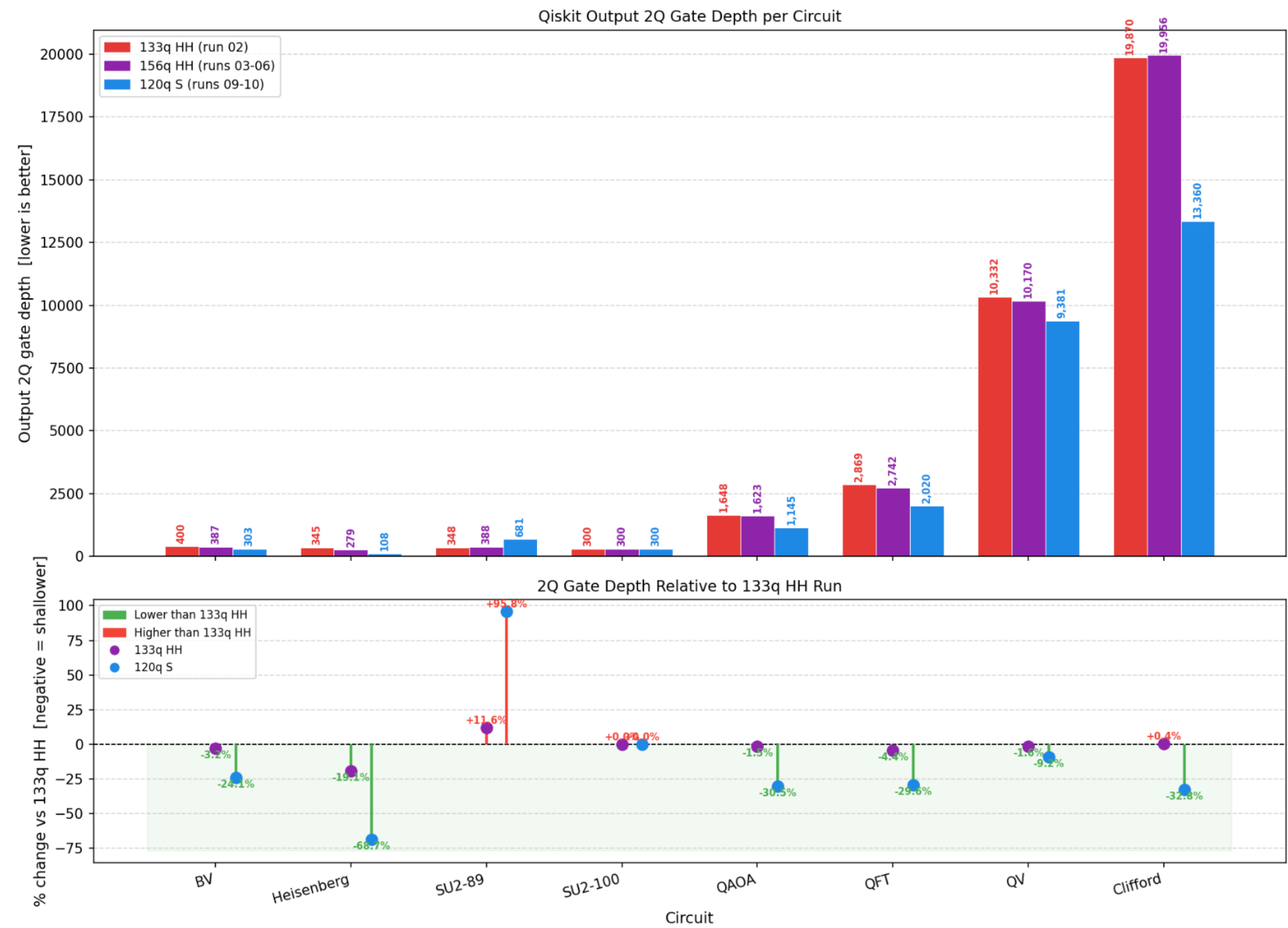


Heavy hex lattice $\longleftrightarrow$ Square lattice

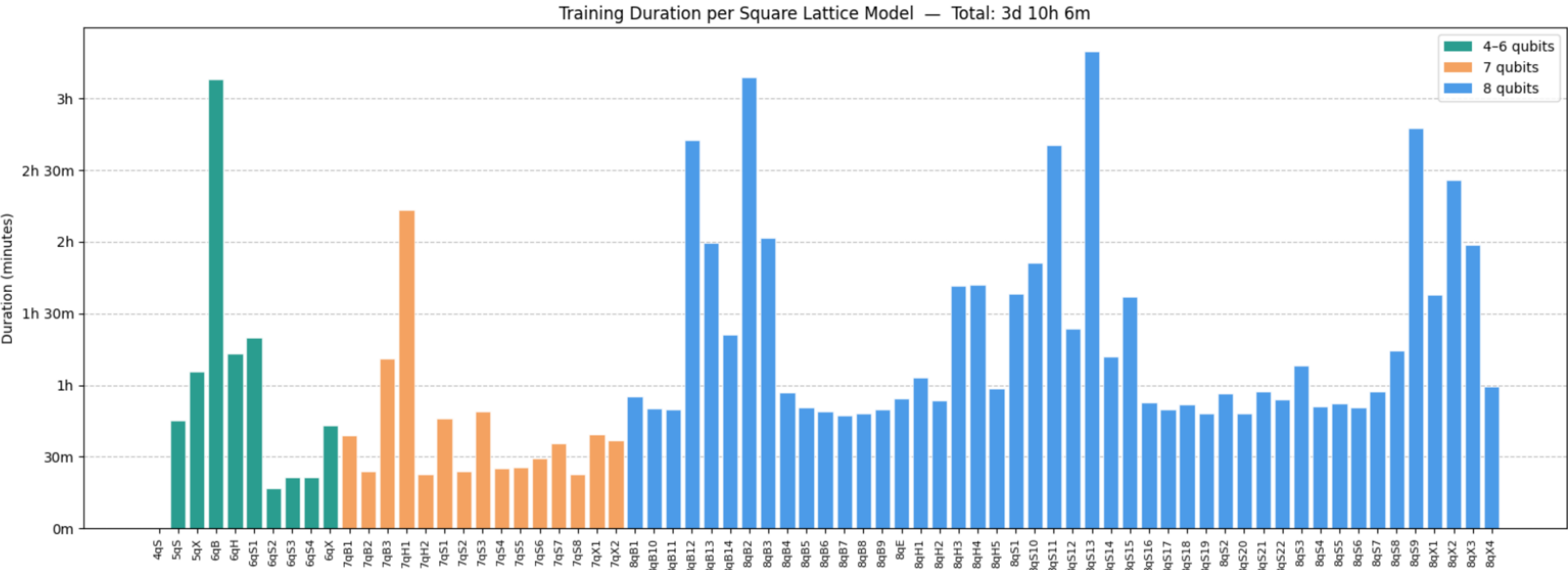
“Can RL-based synthesis be improved by training on subgraphs of a target topology?”

→ Build SQR (square-lattice-trained) and benchmark QPM vs QTS vs SQR

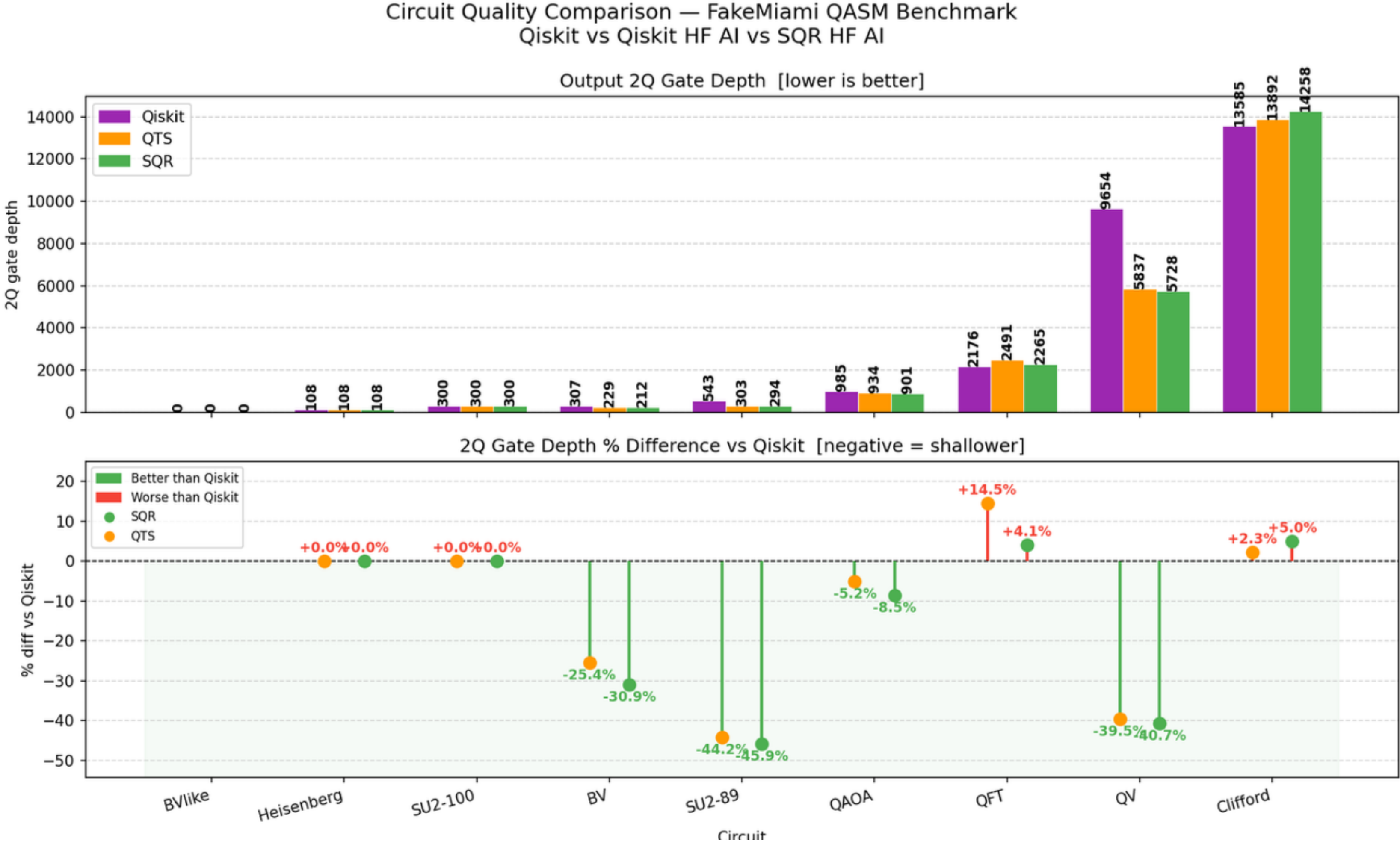
Teaching the AI the new chip



Teaching the AI the new chip



Result: square-trained AI wins (up to 46%)



Open source: squarebench

github.com/DuruTmr444/squarebench

- ① Benchmark framework (QPM vs QTS vs SQR), built on Benchpress
- ② 69 trained square-lattice RL models
- ③ The retraining pipeline

First empirical case for square-lattice RL retraining



Scan for the squarebench repository

<https://github.com/DuruTmr444/squarebench>

Thank you



Scan for the slides & demos

<https://github.com/DuruTmr444/transpilation-talk>

Slides, both demos & README: github.com/DuruTmr444/transpilation-talk